EXPERIMENT 2

Aim

Socket Programming for Transport Layer Packets (TCP Server)

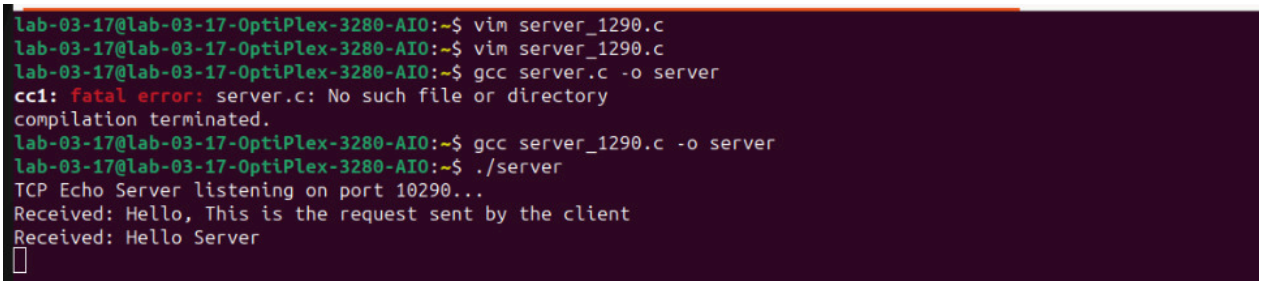
Theory

A **socket** is a software abstraction representing an endpoint of a two-way communication link between two programs running on a network. It is uniquely identified by the combination of an **IP Address** and a **Port Number**.

At the Transport Layer of the OSI model, TCP (Transmission Control Protocol) is used for connection-oriented, reliable, ordered, and error-checked data delivery. TCP Socket Programming on the server side operates as a **Passive Open** process, waiting for incoming connection requests from client nodes.

Server-Side Execution Steps:

1. **Socket Creation (socket()):** The server creates an endpoint for network communication using the domain AF_INET (IPv4) and socket type SOCK_STREAM (TCP).
2. **Binding (bind()):** The created socket is bound to a specific local IP address and port number using the bind() call. This assigns a designated location where clients can direct their connection requests.
3. **Listening (listen()):** The server places the socket into a passive listening state using listen(), maintaining a backlog queue for incoming client connection requests.
4. **Accepting Connection (accept()):** Upon receiving a request, the server executes accept(). This blocks execution until a client connects, completing the **TCP Three-Way Handshake** (SYN, SYN-ACK, ACK). Once accepted, a *new dedicated socket descriptor* is generated specifically for reading and writing data with that specific client.
5. **Data Transfer (recv() / send() or Streams):** Data exchange occurs reliably over stream sockets using input and output buffers.
6. **Closing Connection (close()):** The server terminates the connection using close(), executing the TCP four-wave termination procedure to release system resources.



```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <netinet/in.h>

#define PORT 10000+290
#define BUF_SIZE 1024

int main() {
        int server_fd, new_socket;
        struct sockaddr_in address;
        char buffer[BUF_SIZE];
        int addrlen = sizeof(address);
        server_fd = socket(AF_INET, SOCK_STREAM, 0);
        address.sin_family = AF_INET;
        address.sin_addr.s_addr = INADDR_ANY;
        address.sin_port = htons(PORT);
        bind(server_fd, (struct sockaddr *)&address, sizeof(address));
        listen(server_fd, 3);
        printf("TCP Echo Server listening on port %d...\n", PORT);

        new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
        while (1) {
                memset(buffer, 0, BUF_SIZE);
                int bytes = read(new_socket, buffer, BUF_SIZE);
                if (bytes <= 0) break;
                printf("Received: %s", buffer);
                send(new_socket, buffer, bytes, 0); // echo back
        }
        close(new_socket);
        close(server_fd);
        return 0;
}
~
~
~
~
~
~
~
~
~
~
"server_1290.c" 34L, 916B
```

Conclusion

The TCP Server socket was successfully implemented at the Transport Layer. The passive open execution sequence (socket, bind, listen, and accept) was executed, establishing a reliable, connection-oriented channel capable of processing incoming client data streams.